How the Graded DSL Works

An embedded, type-level-graded language for safe floating point

November 8, 2026

This note explains, from the ground up, how the little language in `src/graded.rs` works. It assumes no background in programming-language theory. One running example is used throughout: the formula

```
(var(0) * var(0) + var(1) * var(1)).sqrt()
```

which is just the length of a vector, $\sqrt{x^2 + y^2}$. Four questions are answered in turn: what an *embedded DSL* is, what the *type checker* did, in what sense the language is *interpreted*, and the role of *operator overloading*.

1 What an embedded DSL is

A **DSL** (domain-specific language) is a small language built for one narrow job. SQL is one: its job is database queries. Regular expressions are one: their job is text patterns.

There are two ways to make a DSL:

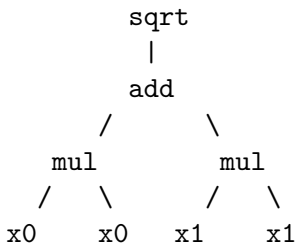
- **External DSL.** You invent brand-new syntax and write a program (a *parser*) that reads that text and makes sense of it. `SELECT * FROM users` is not valid in any ordinary programming language; somebody wrote a parser that understands those words.
- **Embedded DSL.** You do *not* invent new syntax. You write your little language using the ordinary grammar of a *host* language — here, Rust — and simply arrange for the pieces to *mean* something special. It is "embedded" because it lives inside Rust; the Rust compiler reads it as ordinary Rust.

Our language is **embedded**. When you write `var(0) * var(0)`, you are writing one hundred percent legal Rust. There is no parser and no new grammar. The ordinary Rust pieces have just been given a special meaning.

Here is the one idea that unlocks everything else.

> **When you write `var(0) * var(0)`, no multiplication happens.** It *looks* like arithmetic, but nothing is computed. Instead, Rust builds a little model — think of it as writing the recipe step *"multiply these two things"* on an index card, rather than actually multiplying. The whole formula becomes a stack of index cards describing what to do. The actual numbers do not exist yet.

So `(var(0)*var(0) + var(1)*var(1)).sqrt()` does not produce a number. It produces a little **tree of index cards**:



This tree is called an **AST** (abstract syntax tree) — a fancy name for "the recipe, stored as data." Building it is *all* that happens when you write the formula.

2 What the type checker did

The **type checker** is a part of the Rust compiler. It runs *before your program ever executes*, when you build it. Its normal job is to catch mismatches — for instance, making sure you do not try to add a number to a piece of text. Every value has a **type** (a label saying what kind of thing it is: `Number`, `Text`, and so on), and the type checker makes sure the labels are consistent.

Here is the trick. **A real number was hidden inside the type label.**

Normally a label just says "this is a Number." Our labels say "this is an expression *of degree 2*" or "*of degree 0*." The *degree* — the homogeneity weight, i.e. how the value scales when you scale the inputs — is baked into the type itself. And the labels carry arithmetic rules:

- `var(0)` is labelled **degree 1**.
- **Multiplying** two expressions makes the result's degree the **sum** of the two: degree 1 times degree 1 gives degree **2**.
- **Adding** requires both sides to have the **same** degree, and keeps it.
- **`sqrt` halves** the degree — and is only allowed when the degree is even.

The striking part: because these rules live in the *labels*, the type checker is forced to *do the degree arithmetic while it checks your code*, at compile time, before anything runs. Verifying that `var(0)*var(0)` is well-typed literally computes $1 + 1 = 2$. Checking the `.sqrt()` checks that 2 is even and computes $2 \div 2 = 1$. The compiler has been turned into a little calculator for degrees.

Two consequences, both at build time, not run time:

- The formula above comes out labelled **degree 1**. Later, the evaluator reads that label to know how to rescale safely — the degree was computed by the compiler, for free, with zero runtime cost.
- A formula whose degrees do not fit — such as `x*x + 5`, which adds degree 2 to degree 0 — violates the rule for `+` (both sides must match), and **the program refuses to compile.** Not a crash when it runs: it never builds, exactly as if you had tried to add a number to text.

A label like "degree 2" is normally static decoration. Here it is a *number the compiler adds, halves, and compares.* Most languages use types only as coarse categories ("it's a Number"); this DSL uses the type as a *quantity the compiler does mathematics on.* The payoff: the compiler proves a formula is scale-safe-able before it ever runs, and throws out the formulas

that are not — with no runtime check and no way to forget it.

3 In what sense it is interpreted

There are two ways to actually *run* a language.

- **Compiled.** Translate the whole recipe, once, into raw machine instructions the processor runs directly. Afterwards the recipe is gone; only the machine code remains. Fast.
- **Interpreted.** Keep the recipe around as data, and have a small program — the *interpreter* — read it step by step, each time you want an answer, doing what each step says.

Our DSL is **interpreted**. Recall the tree of index cards. When you finally ask for a number — `norm.eval_safe(&[3.0, 4.0])` — a function walks that tree: *"the top card says* ***`sqrt`****, so I first need the card below; it says* ***`add`****, so I need its two children; this one says multiply* ***`x0`*** *by* ***`x0`****, so look up* $x_0 = 3$*, multiply, get* 9 *…"* and so on back up. That walking-the-tree-and-doing-what-it-says is interpretation. The tree stays around; every evaluation re-walks it. The formula is *never* turned into dedicated machine code of its own.

One honest subtlety: the interpreter itself *is* ordinary Rust, which Rust compiles to machine code. So there is compiled code involved — but it is the code of the *interpreter*, not of your *formula*. Distinguish "the language the interpreter is written in" (compiled Rust) from "the little language the interpreter runs" (your formulas, interpreted). The `compile_safe` function, despite its name, does not compile anything: it just prints the recipe as text.

4 The role of operator overloading

The last piece: how is `var(0) * var(0)` even *allowed* to mean "build a multiply card" instead of "multiply"?

Operator overloading means you get to decide what a symbol like `*` or `+` does *when its operands are your own type*. Normally `*` means "multiply two numbers," hardwired. Rust (like C++, Python, and others) instead lets you say: "when someone writes `a * b` and `a`, `b` are my `Expr` type, here is what to do instead." What we told Rust to do for `*` was, in plain words:

> "Do not multiply. Build a new *multiply* index card pointing at the two operands. And set the result's degree label to the *sum* of their degrees."

In code it is barely more than that:

```
impl Mul for Expr {              // "here is what * means for my Expr type"
    fn mul(self, rhs) -> ... {
        Expr::new(Node::Mul(self, rhs))   // build a card; do NOT compute
    }
}
```

Operator overloading is doing two jobs here:

1. **Ergonomics.** It lets you write `x*x + y*y` with ordinary infix symbols instead of ugly nested calls like `add(mul(x,x), mul(y,y))`. The formula reads like mathematics. In most languages this nicer syntax is the *only* reason overloading exists.
2. **The hook.** Because we wrote the body of `*`, that body is exactly where both behaviours were inserted: "build a card" (so nothing computes yet) and "add the degrees" (the label arithmetic the type checker later enforces). Every overloaded operator is a little hook where the DSL's meaning is attached.

5 Where this pays off: the matrix operations

The whole point of the machinery is that real linear-algebra routines should read like their textbook definitions, with the safe rescaling generated rather than typed by hand. There is one new idea needed to get there.

5.1 One scaling knob becomes several

The scalar DSL had a single degree because there was one way to scale the inputs: multiply *everything* by 2^k. But an inner product $\langle x, y\rangle = \sum_i x_i y_i$ has *two independent* knobs — you can scale the x vector and the y vector separately — and the result scales by the product. So the label now carries a *pair* of degrees, $\langle D_x, D_y\rangle$. This is called **multigrading**.

- `xvar()` (the x vector) is labelled $\langle 1, 0\rangle$; `yvar()` is $\langle 0, 1\rangle$.
- Their elementwise product is $\langle 1, 1\rangle$ (the two labels add, componentwise).
- `sum(…)` contracts a vector to a scalar and keeps the label.
- Adding requires both labels to match — so `xvar() + yvar()` (that is $\langle 1, 0\rangle + \langle 0, 1\rangle$) is a **compile error**: you may not add an x-vector to a y-vector.

5.2 Inner product: before and after

Written by hand, the safe inner product carries the scaling openly — find each block's exponent, rescale every term, transport the sum back:

```
let ex = block_exp(maxabs(x));
let ey = block_exp(maxabs(y));
let mut s = 0.0;
for i in 0..n { s += ldexp(x[i], -ex) * ldexp(y[i], -ey); }
ldexp(s, ex + ey)
```

In the DSL the same operation is the naive formula. The label $\langle 1, 1\rangle$ is inferred by the compiler, and `eval_safe` generates the rescaling:

```
let expr = sum(xvar() * yvar());   // bidegree <1,1>, inferred
expr.eval_safe(x, y)               // the block_exp/ldexp is generated
```

`sum(x * y)` is simply the definition of an inner product.

5.3 The other three

Matrix multiply is "each entry is the inner product of a row and a column," delegating to the DSL inner product:

```
for i in 0..rows {
    for j in 0..cols {
        c[i][j] = sum(xvar()*yvar()).eval_safe(row_i(A), col_j(B));
    }
}
```

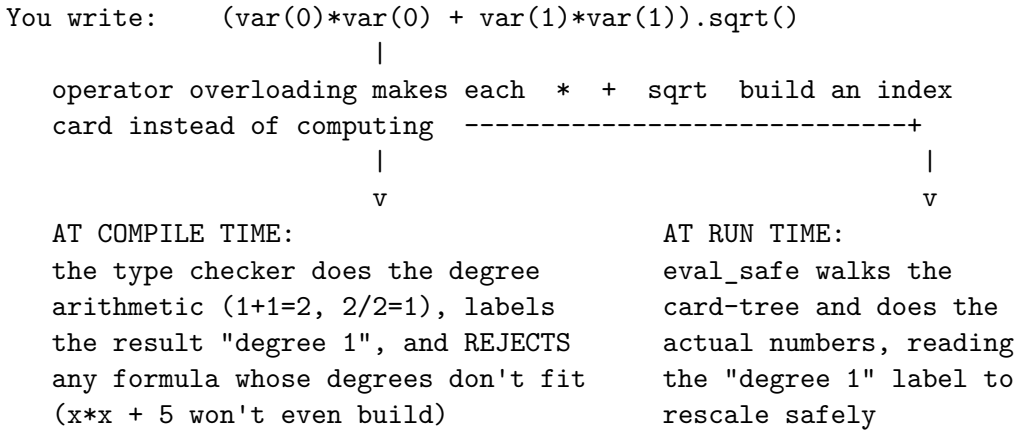
Gram–Schmidt and **eigenvalues (Jacobi)** are *algorithms*, not single formulas — they have loops and "repeat until converged," which no one expression can capture. They stay textbook by *delegation*: every step calls a safe primitive (`dot`, `nrm2`), so no scaling appears in them either. Gram–Schmidt, in full:

```
for j in 0..n {
    let mut w = v[j];
    for i in 0..j { let r = dot(q[i], w);  w -= r * q[i]; }  // remove projection
    let nrm = nrm2(w);
    if nrm > 0 { w /= nrm; }                                 // normalise
    q[j] = w;
}
```

> Count the scaling operations (`ldexp`) visible in the four routines: **zero, zero, zero, and two.** Inner product and matrix multiply have none — the grading generated it. Gram–Schmidt has none — it delegates to safe primitives. Only Jacobi has two lines (rescale once at the top, transport the eigenvalues once at the bottom), because an iterative algorithm cannot be a single graded formula. Every routine now reads the way you would write it on a whiteboard.

There are, in fact, *two* routes to "looks naive but is safe," and the matrix layer uses whichever fits. Inner product and matrix multiply are safe by **grading**: you wrote the formula, a degree in the type generated the rescale. Gram–Schmidt and Jacobi are safe by **composition**: you wrote the algorithm, and it inherits safety from the primitives it calls — which, not coincidentally, is how LAPACK itself is organised.

6 Putting the four together



- **Embedded DSL:** a little language made of ordinary Rust pieces that *build a recipe* instead of computing.
- **Operator overloading:** the mechanism that lets `* + /` build recipe-cards (and carry the degree rules) instead of doing arithmetic.
- **Type checker:** a compile-time calculator that runs the degree arithmetic on the labels and throws out formulas that cannot be made safe — all before the program runs.
- **Interpreted:** at run time, a function walks the recipe-tree step by step to get the number; your formula is never turned into its own machine code.